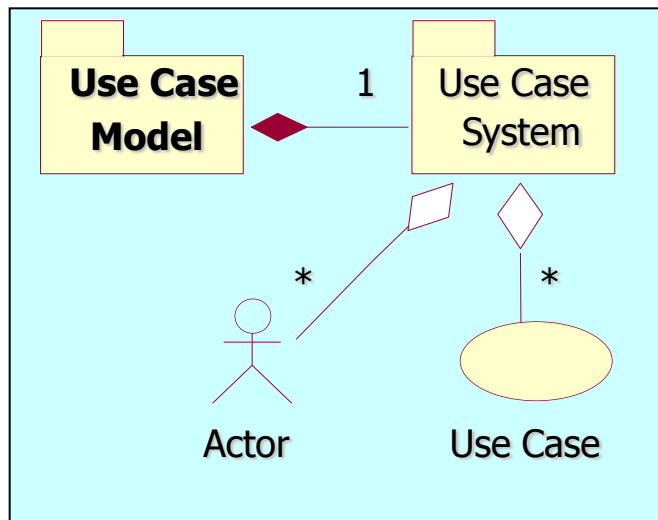


Use-case-driven development

- Use-case-driven development is a key feature of the unified process [BRJ99,JBR99,Kru03].
 - For large projects, it is usually employed as part of an iterative process.
- The use-case-driven development methodology comprises the following basic (activities or) workflows, each producing a corresponding model described by using the UML:
 - Requirements
 - Analysis
 - Design
 - Implementation
 - Test

Use-case-driven development

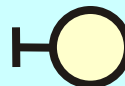
Workflows & models [BRJ99,JBR99,Kru03]:



Requirements

Analysis stereotypes

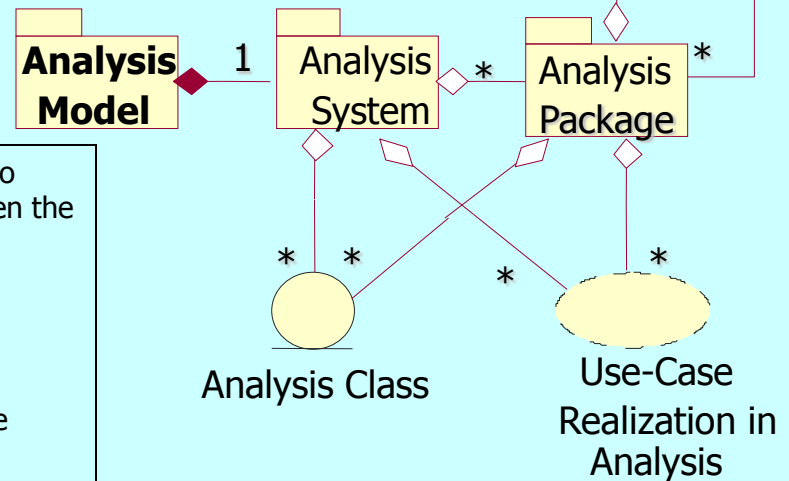
- Boundary class – used to model interactions between the system and its actors



- Control class – used to coordinate (or control) the activities of other classes



- Entity class – used to model long-lived (often persistent) information

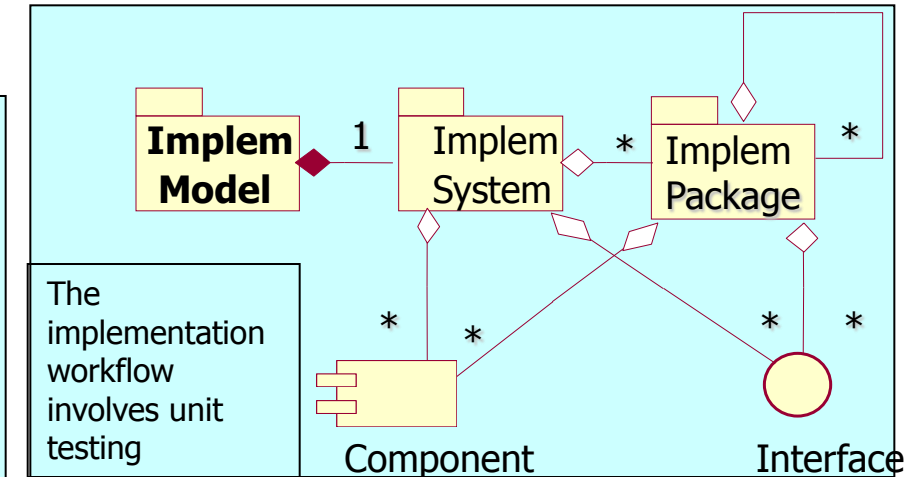


- Typically, there is one control class per use case realization.
- Communication diagrams are primarily used to model the interactions between analysis objects.

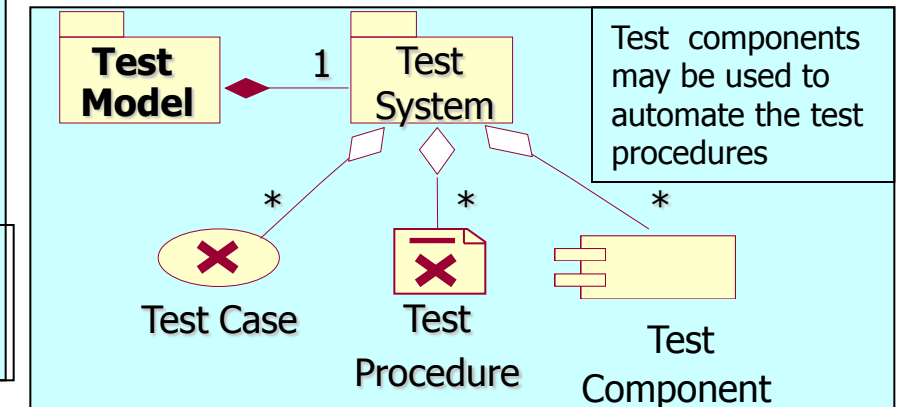
Analysis

[illegible]

Design



Implementation

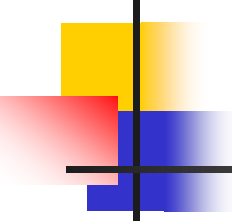


Test



Use-case-driven development

Workflow	Description
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements. The use-case model captures the functional requirements. Specific non-functional requirements can be attached to each use case.
Analysis	A conceptual model of the system based on analysis classes and use-case realizations is created. Conceptual classes are analyzed based on the roles (responsibilities, attributes) they play in the various use-case realizations. An analysis class is an abstraction of a number of design classes.
Design	The design model consists of design classes and sub-systems, use-case realizations, and deployment architectural models. The design model is developed based on the analysis model, but it is also adapted to the implementation environment (GUI kits, architectural frameworks, etc.).
Implementation	The components in the system (that implement design classes and may refine the deployment model) are structured into implementation sub-systems.
Test	System testing follows the completion of the implementation. The test model comprises test cases that can be derived directly from the use case model, test procedures and test components.



References

- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3rd edition). Addison-Wesley, 2003.
- [RS13] D. Rosenberg, M. Stephens, *Use Case Driven Object Modeling with UML: Theory and Practice*, Apress, 2013.
- [Som16] I. Sommerville. *Software Engineering*, (10th edition). Addison-Wesley, 2016.